

Projet F1

Guide des notions importantes

Python · Java · JavaScript

À quoi sert ce projet ?

Le projet prend l'export brut d'un championnat de F1 et le transforme en classements lisibles dans une page web. Chaque langage a un rôle précis, et le résultat de l'un devient l'entrée du suivant.

```
donnees/resultats.csv
  ↓ Python : lire et nettoyer
02-java/courses_propres.csv
  ↓ Java : calculer et trier
03-js/donnees.js
  ↓ JavaScript : afficher
page de classements
```

Les fichiers `courses_propres.csv` et `donnees.js` sont des contrats : ils fixent les colonnes et les noms de champs attendus par l'étape suivante. Si un format change, le maillon d'après casse.

1. Python — lire, nettoyer, écrire

Le notebook `ingestion.ipynb` contient trois fonctions. Le but est de passer d'un CSV brut à un CSV propre, sans toucher au reste du projet.

Convertir un chrono

Un temps comme 1:33.614 doit devenir 93.614 secondes. On sépare les minutes et les secondes, on convertit en nombres, puis on calcule $\text{minutes} \times 60 + \text{secondes}$. Un temps vide renvoie `None`.

Lire puis écrire

La lecture saute la ligne d'en-tête et coupe chaque ligne avec le point-virgule. Un abandon devient la position 0 et un temps vide. L'écriture recrée exactement l'en-tête du contrat, écrit trois décimales et laisse la cellule vide quand le temps vaut `None`.

Difficultés rencontrées

- Oublier de sauter la première ligne du CSV.
- Prendre la virgule pour le séparateur alors que le fichier utilise le point-virgule.
- Mélanger lecture et écriture dans la même fonction.
- Écrire `None` dans le fichier au lieu d'une cellule vide.
- Se tromper sur l'indentation ou sur un espace dans le texte du chrono.

Ma logique

J'ai séparé les trois responsabilités : convertir un temps, lire un fichier, écrire un fichier. Chaque fonction peut être testée seule, et la dernière ne fait que suivre l'ordre du contrat.

Résultat : 4 tests sur 4 passent.

2. Java — calculer les classements

Java reçoit le CSV propre. La classe `Ligne` représente une ligne, `Resultat` représente un pilote ou une écurie.

Le barème

Le tableau `BAREME` donne les points des dix premiers : 25, 18, 15, 12, 10, 8, 6, 4, 2, 1. Une position 0 (abandon) ou au-delà de la 10e place vaut 0.

Classement des pilotes

Je parcours toutes les lignes, je regroupe les résultats par pilote, j'additionne les points et je compte les victoires et les deuxièmes places. Le tri se fait sur quatre critères : points, victoires, deuxièmes places, puis nom.

Ce qui reste

Deux fonctions sont encore à finir : `classementEcuries` (additionner les statistiques des pilotes d'une même écurie) et `positionMoyenne` (moyenne des positions terminées, sans les abandons). Les tests 3 et 4 échouent à cause d'elles, et le test 5 les utilise aussi : d'où le score de 2/5.

Difficultés rencontrées

- Confondre la position 1 avec l'index 0 dans le tableau.
- Comparer des objets au lieu de comparer des nombres.
- Oublier un critère de départage.
- Compter un abandon dans une moyenne.
- Modifier une liste pendant qu'on la parcourt.

Ma logique

J'ai commencé par le barème, puis par le classement des pilotes, car c'est la base du reste. Pour le tri, je compare d'abord les points, puis seulement en cas d'égalité les autres critères.

3. JavaScript — afficher les résultats

Le fichier `donnees.js` contient les classements calculés par Java. JavaScript ne recalcule rien : il trie, crée les lignes du tableau et met le podium en valeur.

Trier sans casser les données

`sort()` modifie la liste reçue. Je trie donc une copie avec `[...liste]`. Le comparateur renvoie la différence entre les points, puis entre les victoires.

Remplir le tableau

Je vide le corps du tableau, puis je crée une ligne et cinq cellules par entrée : rang, nom, écurie, points, victoires. Le nom est aussi stocké dans `data-nom`. Ensuite, les trois premières lignes reçoivent la classe CSS `podium`.

Difficultés rencontrées

La syntaxe est vérifiée, mais les cinq tests doivent encore être lancés dans le navigateur. Je ne mets pas 5/5 tant que je ne les ai pas vus passer.

- Oublier que `sort()` modifie la liste d'origine.
- Trier des nombres sans comparateur numérique.
- Oublier de vider le tableau avant de le remplir à nouveau.
- Confondre une valeur JavaScript et un élément HTML.

Ma logique

J'ai gardé les fonctions petites : une pour trier, une pour remplir, une pour le podium. L'affichage complet se contente de les appeler dans le bon ordre.

4. Tester et avancer

Un test sert à vérifier une règle précise. Quand il échoue, il affiche ce qui était attendu et ce qui a été obtenu. La méthode qui marche le mieux pour moi :

- lancer le premier test en échec ;
- lire l'écart entre attendu et obtenu ;
- afficher une valeur intermédiaire ;
- corriger une seule cause ;
- relancer toute la suite.

Un test peut dépendre de plusieurs fonctions : c'est le cas du test 5 en Java, qui utilise le classement des pilotes, celui des écuries et la position moyenne.

5. Les extensions

- E1 : nettoyer des données sales (casse, accents, doublons, virgules).
- E2 : ajouter le point du meilleur tour, réservé aux dix premiers.
- E3 : filtrer par écurie et trier en cliquant sur les colonnes.
- E4 : lancer toute la chaîne en une commande.

6. Ce que je retiens

- Chaque étape doit produire un fichier au format attendu par la suivante.
- Un contrat clair évite beaucoup d'erreurs entre deux langages.
- Une fonction courte est plus facile à tester et à corriger.
- Un test qui échoue donne souvent l'indice le plus utile.
- Mieux vaut un projet incomplet mais honnête qu'un projet présenté comme fini.

État actuel : Python validé, Java en cours (2/5), JavaScript prêt à être testé dans le navigateur.